



Esercitazione 11

Filtraggio e campionamento di immagini



Introduzione

Questa esercitazione studia due aspetti fondamentali del processamento delle immagini: il **filtraggio** (per ridurre rumore o estrarre informazioni) e il **campionamento** (down/up-sampling).

- **Contesto teorico** — Le immagini digitali contengono componenti a bassa e alta frequenza. Il rumore tipicamente si manifesta come contenuto ad alta frequenza: applicando un filtro passa-basso (es. media mobile o Gaussian blur) si attenuano le componenti ad alta frequenza riducendo il rumore, ma si smussano anche i bordi.
- **Filtri lineari vs non lineari** — Il filtro 5x5 medio usato nell'esercizio è un semplice filtro lineare che uniforma i pixel; è efficace per rumore gaussiano o a bassa entità ma **non** preserva i bordi. Filtri come il mediano o bilateral preservano meglio i dettagli in presenza di rumore impulsivo o quando si vuole conservare bordi.
- **Rilevamento dei bordi** — I filtri differenziali monodimensionali mostrati (es. $[+1/2, 0, -1/2]$ e il suo trasposto) approssimano la derivata spaziale e servono per estrarre componenti ad alta frequenza (bordo/contorni). La magnitudine del gradiente combina le due direzioni e fornisce una mappa di bordo.
- **Campionamento e aliasing** — Il downsampling semplice (prendere ogni 2° riga/colonna) senza prefiltraggio introduce aliasing: alte frequenze si "ripiegano" come basse frequenze, generando artefatti. La soluzione pratica è rispettare il criterio di Nyquist usando un filtro anti-alias (prefiltering) prima di sottocampionare.
- **Upsampling / Interpolazione** — Replicare pixel (nearest neighbor) è veloce ma produce immagini "a blocchi". Interpolazioni bilineare o bicubica danno risultati più naturali, bilanciando qualità e costo computazionale.



Riassunto Esercitazione 11

Obiettivo: comprendere attraverso esperimenti pratici come filtri lineari (media 5x5), filtri differenziali e operazioni di down/up-sampling influenzino rumore, bordi e qualità dell'immagine.

Take-home messages generali

- Prefiltrare con un passa-basso è essenziale prima del downsampling per evitare aliasing.
- Filtri medi semplici riducono rumore ma introducono blur: c'è sempre un compromesso rumore ↔ nitidezza.
- Derivate spaziali (I_x, I_y) sono strumenti semplici ed efficaci per evidenziare bordi; la magnitudine del gradiente aggrega le informazioni direzionali.
- Upsampling per replica è la soluzione più economica ma degrada la percezione (blocchi); usare interp. bilineare/bicubica per risultati migliori.
- Implementazioni matriciali (D, U) sono utili per capire l'operazione lineare di sampling ma attenzione alle dimensioni e ai bordi (immagini non quadrate o dimensioni dispari).



Image Processing: Filtering, Downsampling & Upsampling

1. Image Denoising

- Il rumore nelle immagini deriva da cause **intrinseche** (sensori, scanner, fotocamere) o **estrinseche** (condizioni ambientali).
- Obiettivo: stimare l'immagine originale.
- Applicazioni: **restauro, tracciamento, registrazione, segmentazione, classificazione.**

2. Denoising con Filtri

- Assumendo rumore gaussiano indipendente dai pixel, si può applicare un **filtro passa-basso**.
- Il rumore è interpretato come **componente ad alta frequenza**.
- Esempio: **media locale tramite convoluzione** con kernel.

3. Image Filtering

- I **kernel** modificano l'immagine in modi diversi (sfocatura, rilevamento bordi, ecc.).
- Programmi di grafica forniscono molti di questi filtri come funzioni di base.
- Sono presentati esempi di kernel (K1, K2, K3) che evidenziano bordi e dettagli.

4. Image Scaling & Downsampling

- Immagini troppo grandi richiedono memoria e tempo, quindi si usa la **sotto-campionatura**.
- Il downsampling elimina righe e colonne → immagine dimezzata.
- Problema: **aliasing**, cioè trasformazione delle alte frequenze in frequenze più basse.
- Per evitarlo: rispettare il **criterio di Nyquist** (frequenza di campionamento $\geq 2 \times$ frequenza massima) e applicare **prefiltraggio passa-basso** (es. filtro gaussiano).

5. Gaussian Pyramid

- Struttura multiscala ottenuta tramite successivo **filtraggio gaussiano e sottocampionamento**.

6. Upsampling & Interpolation

- Per aumentare la dimensione si usano tecniche di **interpolazione**.
- **Nearest neighbor**: veloce ma poco accurato.
- Tecniche più raffinate: **bilineare, bicubica**.



In sintesi, le slide introducono i concetti fondamentali di **filtraggio, riduzione e aumento di risoluzione delle immagini**. Vengono spiegati i rischi (aliasing), le soluzioni (prefiltraggio), e le tecniche base (convoluzione, piramidi gaussiane, interpolazione).

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import scipy.signal as ssi
```

✦ Esercizio 1. Filtering and Denoising

- (a) Leggere l'immagine `peppers.png` (scaricabile dalla cartella del corso).
- (b) Visualizzare l'immagine utilizzando il comando `imshow` con l'opzione `cmap='gray'` per visualizzarla in scala di grigi.
- (c) Considerare il seguente filtro bidimensionale:

$$F_{LO} = \frac{1}{25} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Applicare il filtro F_{LO} all'immagine utilizzando la convoluzione bidimensionale (utilizzare il comando `convolve2d` della libreria `scipy.signal`). Visualizzare l'immagine ottenuta.

(d) Creare una matrice N della stessa dimensione dell'immagine `peppers.png` contenente rumore gaussiano con media $\mu = 0$ e deviazione standard $\sigma = 0.05$ (utilizzare il comando `np.random.normal`). Visualizzare l'immagine rumorosa $I_n = I + N$.

(e) Utilizzare il filtro definito al punto (c) per effettuare il denoising dell'immagine I_n . Visualizzare il risultato ottenuto.

```
In [ ]: # === Esercizio 1 - Filtering and Denoising ===

# --- (a) Lettura dell'immagine ---
X = np.array(plt.imread('peppers.png'), dtype='float64')

# --- (b) Visualizzazione dell'immagine originale ---
plt.imshow(X, cmap='gray')
plt.title("Original Image")
plt.axis('off')
plt.show()

# --- (c) Creazione e applicazione del filtro F_LO ---
F_lo = (1.0 / 25.0) * np.ones((5, 5)) # Filtro media mobile 5x5
X_lo = ssi.convolve2d(X, F_lo, mode='same')

plt.imshow(X_lo, cmap='gray')
plt.title("Filtered Image (Low-pass 5x5)")
plt.axis('off')
plt.show()

# --- (d) Aggiunta di rumore gaussiano ---
mu, sigma = 0, 0.05
n = np.random.normal(mu, sigma, X.shape) # Rumore gaussiano
X_n = X + n

plt.imshow(X_n, cmap='gray')
plt.title(f"Noisy Image (μ={mu}, σ={sigma})")
plt.axis('off')
plt.show()

# --- (e) Denoising usando il filtro F_LO ---
X_denoised = ssi.convolve2d(X_n, F_lo, mode='same')

plt.imshow(X_denoised, cmap='gray')
plt.title("Denoised Image (Low-pass 5x5)")
plt.axis('off')
plt.show()
```

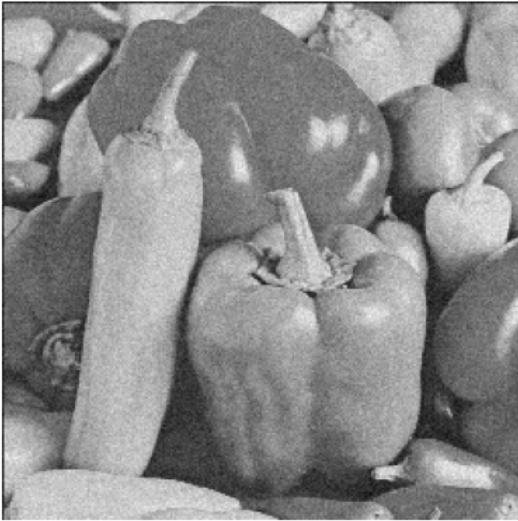
Original Image



Filtered Image (Low-pass 5x5)



Noisy Image ($\mu=0$, $\sigma=0.05$)



Denoised Image (Low-pass 5x5)



COMMENTO.

Obiettivi

- (a) Caricare e visualizzare peppers.png in scala di grigi.
- (c) Applicare un filtro passa-basso 5×5 (media).
- (d) Aggiungere rumore gaussiano ($\mu=0$, $\sigma=0.05$).
- (e) Denoising applicando lo stesso filtro 5×5 all'immagine rumorosa.

Commento sui plot attesi

- *Original Image*: immagine con dettagli netti e texture; contrasti locali evidenti.
- *Filtered Image (Low-pass 5x5)*: aspetto più morbido; dettagli fini e texture attenuate; bordi più smussati. Questo è esattamente l'effetto di una media mobile uniforme.
- *Noisy Image ($\mu=0$, $\sigma=0.05$)*: grana uniforme su tutta l'immagine; aree uniformi diventano "ruvidi"; il contrasto locale rimane ma il SNR cala.
- *Denoised Image (Low-pass 5x5 applied to noisy)*: rumore visibilmente ridotto, ma al costo di ulteriore sfocatura rispetto all'originale → bordi meno netti e possibile perdita di dettagli fini. Se osservi il plot, cerca anche artefatti ai bordi dovuti a come è gestita la convoluzione (mode='same' può introdurre effetti

di bordo se non si usa padding adeguato).

Take-home

- Filtri medi semplici sono facili da implementare e riducono rumore gaussiano a costo di blur.
- Se l'obiettivo è preservare i bordi, preferire filtri pesati (Gaussian), filtri non-lineari (mediano) o tecniche avanzate (bilateral, NL-means).
- Per valutazioni quantitative calcola PSNR o MSE rispetto all'originale (se disponibile la ground truth).

✦ Esercizio 2. Filtraggio e Campionamento

(a) Considerare il seguente filtro monodimensionale:

$$F_{HI-O} = \begin{bmatrix} \frac{1}{2} & 0 & -\frac{1}{2} \end{bmatrix}$$

Applicare il filtro F_{HI-O} all'immagine `peppers.png` e visualizzare l'immagine ottenuta.

(b) Applicare il filtro $F_{HI-V} = F_{HI-O}^T$ all'immagine `peppers.png` e visualizzare l'immagine ottenuta. Qual è la differenza rispetto a quanto ottenuto al punto precedente?

(c) Calcolare

$$|\nabla I| = \sqrt{I_x^2 + I_y^2}$$

dove I_x e I_y sono rispettivamente l'output del filtro al punto (a) e al punto (b). Visualizzare l'immagine ottenuta.

(d) Costruire una matrice D_2 che effettui un sottocampionamento di un fattore 2 ed applicarla all'immagine `peppers.png` e all'immagine filtrata con filtro definito in 1.(c), ovvero calcolare $I_d = D_2 I D_2^T$. Visualizzare i risultati.

(e) Definire un'operazione di sovracampionamento che ad ogni pixel dell'immagine a bassa risoluzione fa corrispondere 4 pixel uguali nell'immagine ad alta risoluzione. Costruire una matrice che effettui l'operazione di sovracampionamento ed applicarla a I_d . Confrontare l'immagine ottenuta con quella originale.

```
In [ ]: # === Esercizio 2 - Filtraggio e Campionamento ===

# Assumo che X sia già caricato dall'esercizio 1 (immagine peppers.png)

# --- (a) Filtro monodimensionale orizzontale ---
F_hild_o = np.array([[+1.0/2.0, 0, -1.0/2.0]]) # Filtro orizzontale
I_x = ssi.convolve2d(X, F_hild_o, mode='same')

plt.imshow(I_x, cmap='gray')
plt.title(r"Filtro  $F_{HI-O}$  ( $I_x$ )")
plt.axis('off')
plt.show()

# --- (b) Filtro monodimensionale verticale ---
F_hild_v = F_hild_o.T # Trasposto del filtro orizzontale
I_y = ssi.convolve2d(X, F_hild_v, mode='same')

plt.imshow(I_y, cmap='gray')
plt.title(r"Filtro  $F_{HI-V}$  ( $I_y$ )")
plt.axis('off')
plt.show()

# --- (c) Magnitudo del gradiente ---
grad_I = np.sqrt(I_x**2 + I_y**2)

plt.imshow(grad_I, cmap='gray')
plt.title(r"Magnitudo del gradiente  $|\nabla I|$ ")
plt.axis('off')
plt.show()

# --- (d) Sottocampionamento di fattore 2 ---
def downsample_matrix(N):
    """Crea matrice di downsampling di fattore 2 per immagini NxN."""
    D = np.zeros((N // 2, N))
    for ii in range(N // 2):
        D[ii, ii * 2] = 1
    return D

# Downsampling immagine originale
D = downsample_matrix(X.shape[0])
I_down = D @ X @ D.T

plt.imshow(I_down, cmap='gray')
plt.title("Downsampling (Original Image)")
plt.axis('off')
plt.show()

# Downsampling immagine filtrata (dall'es.1.c)
D_lo = downsample_matrix(X_lo.shape[0])
I_down_lo = D_lo @ X_lo @ D_lo.T

plt.imshow(I_down_lo, cmap='gray')
plt.title("Downsampling (Filtered Image)")
plt.axis('off')
plt.show()

# --- (e) Upsampling (replicazione pixel) ---
def upsample_matrix(N):
```

```

"""Crea matrice di upsampling che replica i pixel."""
U = np.zeros((N, N // 2))
for ii in range(N // 2):
    U[2 * ii, ii] = 1
    U[2 * ii + 1, ii] = 1
return U

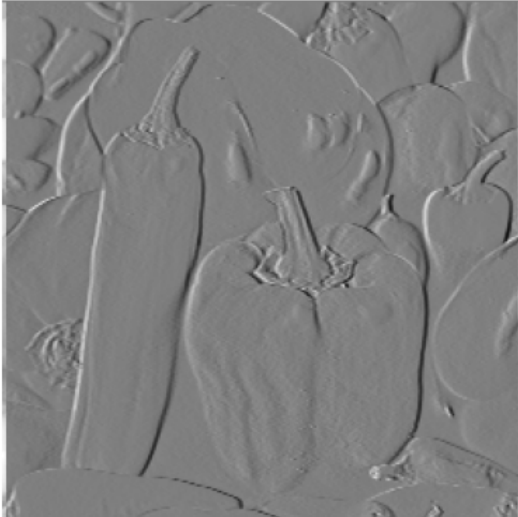
U = upsample_matrix(X.shape[0])

# Upsampling immagine downsampled originale
I_up = U @ I_down @ U.T
plt.imshow(I_up, cmap='gray')
plt.title("Upsampling (Downsampled Original)")
plt.axis('off')
plt.show()

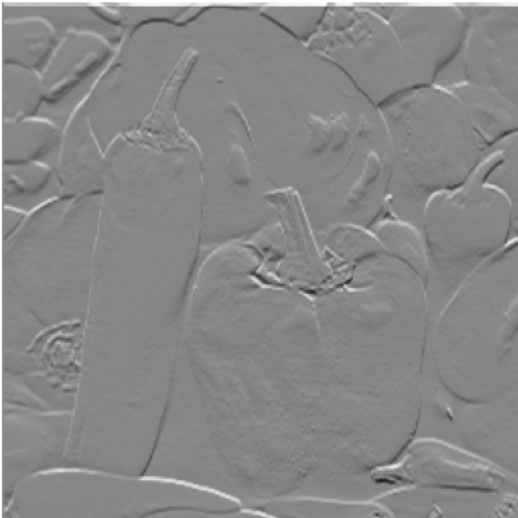
# Upsampling immagine downsampled filtrata
I_up_lo = U @ I_down_lo @ U.T
plt.imshow(I_up_lo, cmap='gray')
plt.title("Upsampling (Downsampled Filtered)")
plt.axis('off')
plt.show()

```

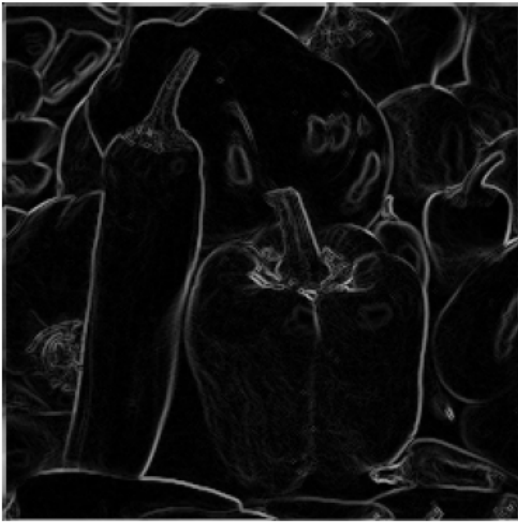
Filtro F_{HI-O} (I_x)



Filtro F_{HI-V} (I_y)



Magnitud del gradiente $|\nabla I|$



Downsampling (Original Image)



Downsampling (Filtered Image)



Upsampling (Downsampled Original)



Upsampling (Downsampled Filtered)



COMMENTO.

Obiettivi

- (a) Applicare filtro monodimensionale orizzontale $F_{HI-O} = [\frac{1}{2}, 0, -\frac{1}{2}]$ (derivata lungo x) $\rightarrow I_x$.
- (b) Applicare filtro verticale $F_{HI-V} = F_{HI-O}^T \rightarrow I_y$.
- (c) Calcolare la magnitudine del gradiente $|\nabla I| = \sqrt{I_x^2 + I_y^2}$.
- (d) Costruire matrice D per downsampling fattore 2 e applicarla all'immagine originale e a quella filtrata (prefilter + downsample).
- (e) Costruire matrice U per upsampling via replicazione (ogni pixel $\rightarrow 2 \times 2$) e applicarla ai risultati downsampliati; confrontare con l'originale.

Commento sui plot

- I_x (filtro orizzontale): mette in evidenza variazioni lungo l'asse x — quindi risponde a **bordi verticali**. Ti aspetti linee luminose/dark dove si hanno transizioni verticali.
- I_y (filtro verticale): evidenzia bordi **orizzontali**.
- Magnitudo del gradiente: mappa con bordi ben delineati (entrambe le direzioni). Spesso il plot va normalizzato per visualizzare correttamente.
- Downsampled (original vs filtered):
 - **Downsampled original (senza prefilter)**: probabili artefatti di aliasing (pattern falsi, perdita di dettagli e shimmer/Moiré in zone ad alta frequenza).
 - **Downsampled filtered (prefilter con 5x5)**: immagine più pulita, senza aliasing pronunciato, ma più sfocata (perdita di dettagli ad alta frequenza).
- Upsampled via replica:
 - Risultato **blocky** (ogni pixel replicato in blocchi 2x2). Se upsampli partendo dal downsampliato **senza** prefilter vedrai blocchi con aliasing; se il downsample era prefiltrato, i blocchi saranno più uniformi e meno "rumorosi".

Take-home

- Derivate discrete lineari sono strumenti semplici ed efficaci per l'estrazione di contorni; la combinazione $I_x/I_y \rightarrow$ magnitudine è la base per molti detector (Sobel, Prewitt).
- **Sempre** applicare un filtro anti-alias (passa-basso) prima di sottocampionare per evitare aliasing indesiderato (Nyquist).
- L'upsampling per replicazione è utile per ricostruzioni molto rapide ma visivamente peggiorative; usare interpolazione per recuperare transizioni più dolci.
- Implementazioni matriciali (D, U) sono didattiche: in pratica si usano funzioni dedicate che gestiscono shape, pad e interpolazioni più robustamente.